

Natural Computing SS23

Exercise Sheet 8 (Mock Exam Solution) – July 13th, 2023

Please mind the following:

- Fill in your **personal information** in the fields below.
- You may use an unmarked dictionary during the exam. **No** additional tools (like calculators, e.g.) might be used.
- Fill in all your answers **directly into this exam sheet**. You may use the backside of the individual sheets or ask for additional paper. In any case, make sure to mark **clearly** which question you are answering. Do not use pens with green or red color or pencils for writing your answers. You may give your answers in both **German and English**.
- Points annotated for the individual tasks only serve as a preliminary guideline.
- At the end of the exam hand in your **whole exam sheet**. Please make sure you do not undo the binder clip!
- To forfeit your exam (“entwerten”), please cross out clearly this **cover page** as well as **all pages** of the exam sheet. This way the exam will not be evaluated and will not be counted as an exam attempt.
- Please place your LMU-Card or student ID as well as photo identification (“Lichtbildausweis”) clearly visible on the table next to you. Note that we need to check your data with the data you enter on this cover sheet.

Time Limit: 90 minutes

First Name:																									
Last Name:																									
Matriculation Number:																									
<table border="1"><tr><td>Topic 1</td><td>max. 10 pts.</td><td>pts.</td></tr><tr><td>Topic 2</td><td>max. 15 pts.</td><td>pts.</td></tr><tr><td>Topic 3</td><td>max. 10 pts.</td><td>pts.</td></tr><tr><td>Topic 4</td><td>max. 20 pts.</td><td>pts.</td></tr><tr><td>Topic 5</td><td>max. 15 pts.</td><td>pts.</td></tr><tr><td>Topic 6</td><td>max. 15 pts.</td><td>pts.</td></tr><tr><td>Topic 7</td><td>max. 5 pts.</td><td>pts.</td></tr><tr><td colspan="2">Total max. 90 pts.</td><td>pts.</td></tr></table>		Topic 1	max. 10 pts.	pts.	Topic 2	max. 15 pts.	pts.	Topic 3	max. 10 pts.	pts.	Topic 4	max. 20 pts.	pts.	Topic 5	max. 15 pts.	pts.	Topic 6	max. 15 pts.	pts.	Topic 7	max. 5 pts.	pts.	Total max. 90 pts.		pts.
Topic 1	max. 10 pts.	pts.																							
Topic 2	max. 15 pts.	pts.																							
Topic 3	max. 10 pts.	pts.																							
Topic 4	max. 20 pts.	pts.																							
Topic 5	max. 15 pts.	pts.																							
Topic 6	max. 15 pts.	pts.																							
Topic 7	max. 5 pts.	pts.																							
Total max. 90 pts.		pts.																							

1 General Knowledge / Multiple Choice

10pts

For each of the following questions select **one** correct answer ('1 of n '). Every correct answer is awarded one point. Multiple answers or incorrect answers will be marked with zero points.

(a) Which of the following disciplines is a common inspiration for natural computing? (1pt)

i Romance	ii <u>Biology</u>	iii History	iv Law
-----------	-------------------	-------------	--------

(b) In the game of life there exist states that have no predecessor, i.e., they can never be generated from another state via the rules of the game of life, but they can be used as initial states for an instance of the game of life. **Such a pattern is called...?** (1pt)

i <u>Garden of Eden</u>	ii Big Bang	iii Noah	iv Atlantis
-------------------------	-------------	----------	-------------

(c) Any computer program can be encoded into an instance of Conway's game of life that performs an equivalent computation. **Thus, Conway's game of life is...?** (1pt)

i <u>Turing-complete</u>	ii von-Neumann-complete	iii Zorn-complete	iv Gabor-complete
--------------------------	-------------------------	-------------------	-------------------

(d) As measured by sample efficiency, all optimization algorithms perform the same when averaged over all possible target functions. **What is this theorem called?** (1pt)

i Cheap Breakfast Theorem	ii <u>No Free Lunch Theorem</u>	iii Pay For Dinner Theorem	iv Secret Midnight Snack Theorem
---------------------------	---------------------------------	----------------------------	----------------------------------

(e) What is commonly not part of a run of an evolutionary algorithm? (1pt)

i a population history	ii <u>a global optimum</u>	iii a fitness function	iv a search space
------------------------	----------------------------	------------------------	-------------------

(f) Why are measures to increase the diversity within the population of an evolutionary algorithm usually employed? (1pt)

i ensure algorithmic greediness	ii ensure quick exploitation	iii avoid early exploration	iv <u>avoid</u> <u>premature</u> <u>convergence</u>
---------------------------------	------------------------------	-----------------------------	---

(g) What is a necessary part of the definition of a neural network? (1pt)

i colors	ii <u>weights</u>	iii biases	iv feelings
----------	-------------------	------------	-------------

(h) Sufficiently large neural networks can...? Pick the strongest true statement! (1pt)

i approximate only linear functions to a fixed degree at best	ii approximate only linear functions to an arbitrary degree	iii approximate any function to a fixed degree at best	iv <u>approximate</u> <u>any function to</u> <u>an arbitrary</u> <u>degree</u>
---	---	--	---

(i) A quantum bit (qubit) can assume a state that when measured collapses to the classical states 0 or 1, but before measuring acts to some extent like a mixture of the states 0 and 1. **This state is called...?** (1pt)

i plusinformation	ii <u>superposition</u>	iii hyperrelaxation	iv ubertunneling
-------------------	-------------------------	---------------------	------------------

(j) Assume that we entangled two qubits so that they can assume the combined states (0,0) and (1,1). We prepare the exact same experiment multiple times and each time measure the first qubit and notice that we receive the state 0 in 25% of experiments and 1 in 75% of the experiments. Each time, we then measure the second qubit after having measured the first. **What results do we observe for the second qubit?** (1pt)

i a state 1 in 0% of the experiments	ii a state 1 in 25% of the experiments	iii <u>a state 1 in</u> <u>75% of the</u> <u>experiments</u>	iv a state 1 in 100% of the experiments
--------------------------------------	--	--	---

2 Game of Life

15pts

The following definition was given in the lecture.

Definition 1 (Conway's game of life (standard)). Let $G = (V, E)$ be a graph with vertices V and (undirected) edges $E \subseteq V \times V$ with a fixed degree of 8 for all nodes. We define $\text{surroundings} : V \rightarrow \wp(V)$ via

$$\text{surroundings}(v) = \{w \mid (v, w) \in E\}$$

so that $|\text{surroundings}(v)| = 8$ and $v \notin \text{surroundings}(v)$ for all $v \in V$. A state $x \in \mathcal{X}$ is a mapping of vertices to the labels $\{\text{dead}, \text{alive}\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \rightarrow \{\text{dead}, \text{alive}\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. We define

$$|v|_{x_t} = |\{w \mid w \in \text{surroundings}(v) \wedge x_t(w) = \text{alive}\}|.$$

In the game of life, the evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via

$$x_{t+1}(v) = \begin{cases} \text{dead} & \text{if } |v|_{x_t} \leq 1, \\ x_t(v) & \text{if } |v|_{x_t} = 2, \\ \text{alive} & \text{if } |v|_{x_t} = 3, \\ \text{dead} & \text{if } |v|_{x_t} \geq 4, \end{cases}$$

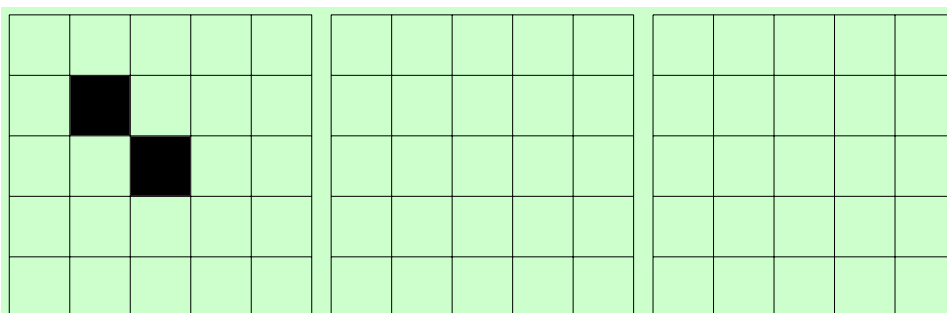
for all $v \in V$. A tuple (G, x_0) is called an instance of the game of life for initial state $x_0 \in \mathcal{X}$.

We now introduce a new formal definition:

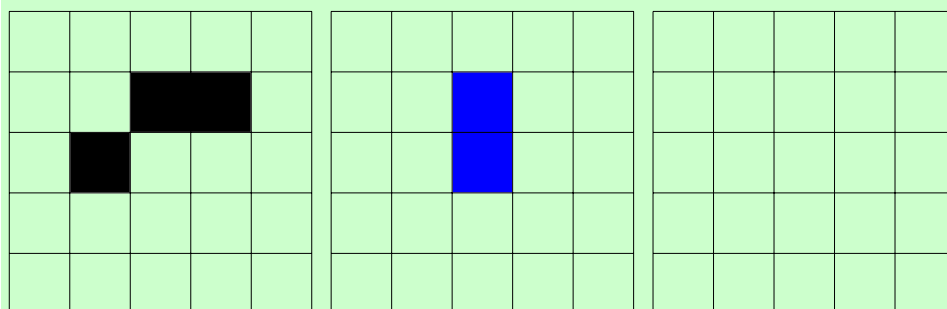
Definition 2 (exam spaceship). Let (G, x_S) be an instance of the game of life with $G = (V, E)$ played on an $m \times n$ grid. Let $x_{S+1}, x_{S+2}, \dots$ be the subsequent states of that instance. (G, x_S) is called an *exam spaceship* if there exists a time frame $\Delta t \in \mathbb{N} \setminus \{0\}$ and there exist displacement values $\Delta x, \Delta y \in \mathbb{Z}$ so that for all $v_{i,j} \in V$ it holds that

$$x_{S+\Delta t}(v_{(i+\Delta y \bmod n), (j+\Delta x \bmod m)}) = x_S(v_{i,j}).$$

(a) **Show why these two non-empty instances of the game of life are not exam spaceships.** You may use the empty grids to the right of the instances to draw the subsequent states of these instances. (5pts)

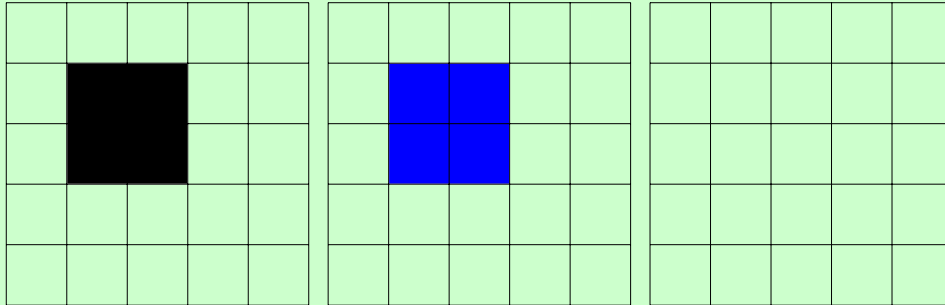


Field dies out at $t = 1$, can never reach initial state or any displacement of it.

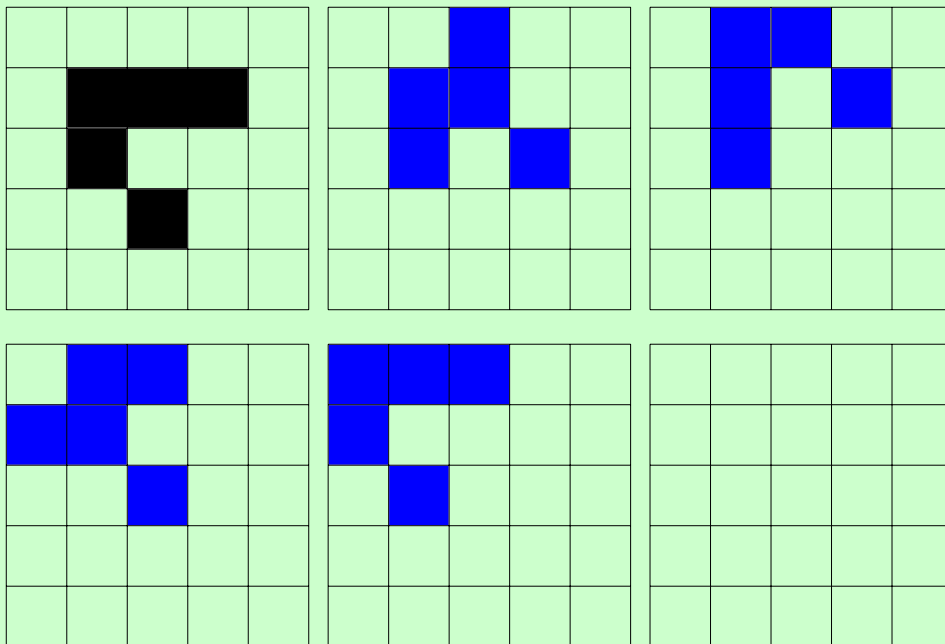


Field dies out at $t = 2$, can never reach initial state or any displacement of it.

(b) Show why these two non-empty instances of the game of life are *indeed* exam spaceships. Also give the time frame Δt and the respective displacement values $\Delta x, \Delta y$ for which they fulfill Definition 2. You may use the empty grids to the right and bottom of the instances to draw the subsequent states of these instances. (7pts)



Exam spaceship for $\Delta t = 1$ with $\Delta x = 0, \Delta y = 0$



Exam spaceship for $\Delta t = 4$ with $\Delta x = -1, \Delta y = -1$

(c) The definition of an exam spaceship does not match the common definition of a spaceship. Commonly, a spaceship does need to travel across the grid and cannot remain in one place. **Give a constraint on Δt , Δx , and/or Δy from Definition 2 to express this property.** (3pts)

$$\neg(\Delta x = 0 \wedge \Delta y = 0)$$

3 Quantum Computing

10pts

The following definitions were given in the lecture.

Definition 3 (quantum state). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{Q}(\mathcal{X})$ be the space of superposition states with bases in $\mathcal{X}$, i.e., each $q \in \mathcal{Q}(\mathcal{X})$ has the form $q = \sum_{x \in \mathcal{X}} c_x |x\rangle$ where $c_x \in \mathbb{C}$ for any $x \in \mathcal{X}$ is called an amplitude so that $\sum_{x \in \mathcal{X}} |c_x|^2 = 1$ and $|x\rangle$ (read “ket x ”) denotes a vector corresponding to the state $x \in \mathcal{X}$.

We use the following terminology:

- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.
- If $\mathcal{X} = \mathcal{X}_A \times \mathcal{X}_B$, i.e., $\mathcal{X}$ can be constructed as a product of two other state spaces $\mathcal{X}_A, \mathcal{X}_B$, then it follows that q has the form $q = \sum_{x_A \in \mathcal{X}_A, x_B \in \mathcal{X}_B} c_{x_A, x_B} |x_A\rangle \otimes |x_B\rangle$ where $\otimes$ is the tensor product. If q cannot be written as $q = (\sum_{x_A \in \mathcal{X}_A} c_{x_A} |x_A\rangle) \otimes (\sum_{x_B \in \mathcal{X}_B} c_{x_B} |x_B\rangle)$, then we call q an *entangled* state.
- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as $q = c_0 |0\rangle + c_1 |1\rangle$ for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.
- If $\mathcal{X} = \{0, 1\}^n$ for any $n \in \mathbb{N}$ (i.e., any $x \in \mathcal{X}$ is a bit register), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit register* of size n and can be written as $q = \sum_{i=0}^{2^n-1} c_i |i\rangle = c_{0\dots 0} |0\dots 0\rangle + \dots + c_{1\dots 1} |1\dots 1\rangle$ where we usually denote i in binary as illustrated above.
- When we *measure* a quantum state $q \in \mathcal{Q}(\mathcal{X})$, we generate classical state $M(q) \in \mathcal{X}$ so that for all $x \in \mathcal{X}$ it holds that $\Pr(M(q) = x) = |c_x|^2$.

(a) Give an example of any valid two-qubit quantum state where exactly one of the qubits is in state $|1\rangle$. (4pts)

General form: $|1\rangle \otimes |\phi\rangle = (0|0\rangle + 1|1\rangle) \otimes (\alpha|0\rangle + \beta|1\rangle)$

The most simple solution would be $|1\rangle \otimes |0\rangle = |10\rangle$ but any multiplication with valid state ϕ would suffice.

Definition 4 (Ising model). Let $G = (V, E)$ be a graph with vertices $V = \{v_0, \dots, v_{n-1}\}$ for $n \in \mathbb{N}$ and undirected edges $E \subseteq V \times V$. A spin configuration $x \in \mathcal{X} = (V \rightarrow \{-1, +1\})$ is a mapping of vertices to values $\{-1, +1\}$. This mapping x is fully defined by a spin vector $\vec{\sigma} \in \{-1, +1\}^n$ so that $x(v_i) = \sigma_i$ for any $i \in \{0, \dots, n-1\}$. The energy of an Ising model is then given via

$$H(\vec{\sigma}) = - \sum_{\substack{i \in \{0, \dots, n-1\} \\ j \in \{0, \dots, n-1\}}} J_{ij} \sigma_i \sigma_j - \sum_{i \in \{0, \dots, n-1\}} h_i \sigma_i$$

where $J \in \mathbb{R}^{n \times n}$ is called the interaction matrix and $\vec{h} \in \mathbb{R}^n$ is called the vector of field strengths. We also assume that for any $i, j \in \{0, \dots, n-1\}$ it holds that $i \geq j \implies J_{ij} = 0$, i.e., J is a strictly upper triangle matrix, and that $(v_i, v_j) \notin E \implies J_{ij} = 0$, i.e., J_{ij} can only have non-zero values where an edge between i and j exists in the substrate graph G .

For an Ising model given by (G, J, h) , there are processes (both artificial and natural) that produce a spin configuration $\vec{\sigma}$ so that $H(\vec{\sigma})$ is minimized.

Definition 5 (QUBO). Let $Q \in \mathbb{R}^{n \times n}$ be an arbitrary matrix called QUBO matrix. The process of finding a vector $\vec{x} \in \mathbb{B}^n$ so to

$$\text{minimize } H(\vec{x}) = x^\top Q x = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} Q_{ij} x_i x_j$$

is called *quadratic unconstrained binary optimization (QUBO)*. W.l.o.g. we can assume that Q is a (non-strictly) upper triangular matrix.

Consider matrix $M = \begin{pmatrix} 0 & 3 \\ & 0 \end{pmatrix}$ and vector $v = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$.

(b) **State if M and v (could) form an Ising or a QUBO formulation? Briefly explain why.** (2pts)

Ising, where M is the interaction matrix and v is the field vector. M could also be a QUBO matrix, but what would the vector be then...?

(c) **Convert the matrix formulation of M in the respective *other* version.** (4pts)

$$\begin{aligned}
H_{\text{Ising}} &= -3\sigma_1\sigma_2 - 2\sigma_1 - 1\sigma_2 && \text{from Ising energy formula} \\
&= -3(2x_1 - 1)(2x_2 - 1) - 2(2x_1 - 1) - 1(2x_2 - 1) && \text{substitute } \sigma_i = 2x_i - 1 \\
&= -12x_1x_2 + 6x_1 + 6x_2 - 3 - 4x_1 + 2 - 2x_2 + 1 \\
&= -12x_1x_2 + 2x_1 + 4x_2 + 0 \\
&=: H_{\text{QUBO}} \\
\Rightarrow Q &= \begin{pmatrix} 2 & -12 \\ & 4 \end{pmatrix} \text{ with offset } 0
\end{aligned}$$

4 Optimization

20pts

We give a shortened version of the definition for optimization processes from the lecture and then introduce a new optimization algorithm called *exam search*.

Definition 6 (optimization (shortened)). Let $\mathcal{X}$ be an arbitrary set called state space. Let $\mathcal{T}$ be an arbitrary set called target space and $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization or minimization is the procedure of searching for a $x \in \mathcal{X}$ so that $\tau(x)$ is optimal or minimal.

An optimization run of length $g + 1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t . Let e be a possibly randomized or non-deterministic function so that the optimization run is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$, where x_0 is given externally (i.e., $x_0 =_{\text{def}} 42$) or chosen randomly (i.e., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

Algorithm 1 (exam search). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. We assume $\mathcal{X} = \mathbb{B}^4$ and $\mathcal{T} = \mathbb{N}$. Let $neighbors : \mathbb{B}^4 \rightarrow \wp(\mathbb{B}^4)$ be given by

$$neighbors(x) = \{(\neg x(0), x(1), x(2), x(3)), (x(0), \neg x(1), x(2), x(3)), \\ (x(0), x(1), \neg x(2), x(3)), (x(0), x(1), x(2), \neg x(3))\}$$

where $\neg$ is the logical NOT function, i.e., $\neg 0 = 1$ and $\neg 1 = 0$. The process $\mathcal{D}$ continues via exam search iff e is given by

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x' \in neighbors(x_t) \cup \{x_t\}} \tau(x').$$

(a) Let target function

$$\tau((x(0), x(1), x(2), x(3))) = 15 - (8 \cdot x(0) + 4 \cdot x(1) + 2 \cdot x(2) + 1 \cdot x(3)).$$

Let initial state $x_0 = (0, 0, 0, 0)$. **Apply exam search to this setting for 5 time steps, i.e., compute $x_1, \dots, x_5$. Does exam search reach the global optimum within these time steps? What percentage of all possible states within $\mathbb{B}^4$ has been evaluated during this optimization run? How can this number be interpreted? (11pts)**

$$\tau(x_0) = 15$$

$$neighbors(x_0) = \left\{ \begin{array}{ll} (1, 0, 0, 0) & \text{with } \tau(\cdot) = 7, \\ (0, 1, 0, 0) & \text{with } \tau(\cdot) = 11, \\ (0, 0, 1, 0) & \text{with } \tau(\cdot) = 13, \\ (0, 0, 0, 1) & \text{with } \tau(\cdot) = 14 \end{array} \right\}$$

$$x_1 = (1, 0, 0, 0) \text{ with } \tau(x_1) = 7.$$

$$neighbors(x_1) = \left\{ \begin{array}{ll} (0, 0, 0, 0) & \text{with } \tau(\cdot) = 15, \\ (1, 1, 0, 0) & \text{with } \tau(\cdot) = 3, \\ (1, 0, 1, 0) & \text{with } \tau(\cdot) = 5, \\ (1, 0, 0, 1) & \text{with } \tau(\cdot) = 6 \end{array} \right\}$$

$$x_2 = (1, 1, 0, 0) \text{ with } \tau(x_2) = 3.$$

$$neighbors(x_2) = \left\{ \begin{array}{ll} (0, 1, 0, 0) & \text{with } \tau(\cdot) = 11, \\ (1, 0, 0, 0) & \text{with } \tau(\cdot) = 7, \\ (1, 1, 1, 0) & \text{with } \tau(\cdot) = 1, \\ (1, 1, 0, 1) & \text{with } \tau(\cdot) = 2 \end{array} \right\}$$

$$x_3 = (1, 1, 1, 0) \text{ with } \tau(x_3) = 1.$$

$$neighbors(x_3) = \left\{ \begin{array}{ll} (0, 1, 1, 0) & \text{with } \tau(\cdot) = 9, \\ (1, 0, 1, 0) & \text{with } \tau(\cdot) = 5, \\ (1, 1, 0, 0) & \text{with } \tau(\cdot) = 3, \\ (1, 1, 1, 1) & \text{with } \tau(\cdot) = 0 \end{array} \right\}$$

$$x_4 = (1, 1, 1, 1) \text{ with } \tau(x_4) = 0.$$

$$neighbors(x_4) = \left\{ \begin{array}{ll} (0, 1, 1, 1) & \text{with } \tau(\cdot) = 8, \\ (1, 0, 1, 1) & \text{with } \tau(\cdot) = 4, \\ (1, 1, 0, 1) & \text{with } \tau(\cdot) = 2, \\ (1, 1, 1, 0) & \text{with } \tau(\cdot) = 1 \end{array} \right\}$$

$$x_5 = x_4 = (1, 1, 1, 1) \text{ with } \tau(x_5) = 0.$$

The global optimum was found. Exam search evaluated 14/16 states, which almost covers the whole search space (like brute force would).

- (b) Give a target function $\tau : \mathbb{B}^4 \rightarrow \mathbb{N}$ for which exam search never reaches the global optimum when starting from $x_0 = (0, 0, 0, 0)$. Briefly explain why. (5pts)

$$\tau(x) = \begin{cases} 0 & \text{if } x = (1, 1, 1, 1), \\ 1 & \text{if } x = (0, 0, 0, 0), \\ 2 & \text{otherwise.} \end{cases}$$

Exam search never escapes the local optimum at $(0, 0, 0, 0)$.

- (c) State if exam search is elitist. Briefly explain why. (2pts)

Exam search is elitist because it never lets go of its currently best solution candidate, which is always included within the range of the arg min operator in function e .

- (d) State if exam search is greedy. Briefly explain why. (2pts)

Exam search is greedy because it *always* moves on to a better solution once discovered by the nature of the arg min operator in function e .

5 Artificial Chemistries / Soups

15pts

We used the following definition in the lecture:

Definition 7 (artificial chemistry, soup). Let $\mathcal{X}$ be a state space. Let $\mathcal{R}$ be a set of reaction rules $R \in \mathcal{R}$ where each R is a function $R : \mathcal{X}_R^{\text{dom}} \rightarrow \mathcal{X}_R^{\text{cod}}$ for $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$. Let $\mathcal{A}(X, \mathcal{R})$ be the set of all *reaction materials* from $X \in \wp^*(\mathcal{X})$ for all rules in $\mathcal{R}$, i.e.,

$$\mathcal{A}(X, \mathcal{R}) = \{(X', R) \mid X' \subseteq X \wedge X' \in \mathcal{X}_R^{\text{dom}} \wedge R \in \mathcal{R}\}.$$

Let $A : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X}) \times \mathcal{R}$ be a possibly randomized or non-deterministic function that takes a multi-set $X \in \wp^*(\mathcal{X})$ and returns a multi-set of reactants and a reaction rule to perform so that $A(X) \in \mathcal{A}(X, \mathcal{R})$. The tuple $(\mathcal{X}, \mathcal{R}, A)$ defines an artificial chemistry. A multi-set $X \in \wp^*(\mathcal{X})$ of an artificial chemistry is also called soup; each element $x \in X$ is also called a particle. The evolution of a soup for g generations is a sequence $\langle X_0, \dots, X_g \rangle$ so that X_0 is given as the initial soup and, for all t with $0 \leq t \leq g-1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where $(X', R) = A(X_t)$.

Reaction rules are usually written in the form $R = \sum_i x_i \rightarrow \sum_j y_j$ for $x_i, y_j \in \mathcal{X}$, $i, j \in \mathbb{N}$, $\{x_i\}_i \in \mathcal{X}_R^{\text{dom}}$, $\{y_j\}_j \in \mathcal{X}_R^{\text{cod}}$, and $R(\{x_i\}_i) = \{y_j\}_j$.

For the following tasks we define an artificial chemistry $\mathcal{B} = (\mathcal{X}, \mathcal{R}, A)$ where $\mathcal{X} = \{\text{EGG}, \text{CATERPILLAR}, \text{COCOON}, \text{BUTTERFLY}, \text{TREE}, \text{SQUIRREL}\}$ and $\mathcal{R} = \{R_1, \dots, R_6\}$ where

$$R_1 = \text{EGG} \rightarrow \text{CATERPILLAR},$$

$$R_2 = \text{CATERPILLAR} + \text{TREE} \rightarrow \text{COCOON} + \text{TREE},$$

$$R_3 = \text{COCOON} \rightarrow \text{BUTTERFLY},$$

$$R_4 = \text{BUTTERFLY} + \text{BUTTERFLY} \rightarrow \text{EGG} + \text{EGG} + \text{EGG} + \text{EGG},$$

$$R_5 = \text{SQUIRREL} + \text{EGG} \rightarrow \text{SQUIRREL},$$

$$R_6 = \text{SQUIRREL} + \text{CATERPILLAR} \rightarrow \text{SQUIRREL}.$$

For the function A we assume that it first picks a uniform random rule $R \in \mathcal{R}$ and then checks if all particles necessary to apply R exist within the current soup X_t at time t . If such particles exist, A picks a random set of suitable particles X' to match the left hand side of rule R and then returns (X', R) . Whenever suitable particles for R do not exist within X_t , A again picks a uniform random rule $R \in \mathcal{R}$ and tries to use that.

(a) Give a possible evolution of the soup

$$X_0 = \{\text{CATERPILLAR}, \text{BUTTERFLY}, \text{TREE}\}$$

for as many time steps as necessary until an EGG appears. (Hint: Instead of random choices, you can decide which rules to apply to make the process quicker. Use the table below.) (3pts)

time step	chosen rule	current soup
0	none	{CATERPILLAR, BUTTERFLY, TREE}
1	R_2	{TREE, BUTTERFLY, COCOON}
2	R_3	{TREE, BUTTERFLY, BUTTERFLY}
3	R_4	{TREE, EGG, EGG, EGG, EGG}
4		
5		

(b) Give initial populations $X_0^{stay}, X_0^{grow}, X_0^{shrink}$ so that when we evolve these populations separately, i.e., we compute $X_t^{stay}, X_t^{grow}, X_t^{shrink}$, respectively, for increasingly large values of t by applying the artificial chemistry $\mathcal{B}$, it holds that $|X_t^{stay}|$ remains constant for all t , $|X_t^{grow}|$ eventually increases with sufficiently large t , and $|X_t^{shrink}|$ eventually shrinks with sufficiently large t . (3pts)

$$\begin{aligned} X_0^{stay} &= \emptyset \\ X_0^{grow} &= \{\text{BUTTERFLY}, \text{BUTTERFLY}\} \\ X_0^{shrink} &= \{\text{SQUIRREL}, \text{EGG}\} \end{aligned}$$

(c) For this task, consider a soup of artificial chemistry $\mathcal{B}$ with the initial population

$$X_0 = \{\text{EGG}, \text{SQUIRREL}, \text{TREE}\}.$$

What are the chances that anywhere in the future evolution of X_0 a BUTTERFLY appears? Show your reasoning. You can assume that all executions of A eventually terminate. You can draw all possible paths for the evolution of X_0 to derive the chances. (5pts)

Hint: Review exactly which behavior A uses.

```

{EGG, SQUIRREL, TREE}
|  [R1], R5          1/2 chance
{CATERPILLAR, SQUIRREL, TREE}
|  [R2], R6          1/2 chance
{COCOON, SQUIRREL, TREE}
|  [R3]              1/1 chance
{BUTTERFLY, SQUIRREL, TREE}

```

Since this is the only path, the overall chances of a BUTTERFLY appearing are $1/2 \cdot 1/2 \cdot 1 = 1/4$.

(d) For this task, consider another soup of artificial chemistry $\mathcal{B}$ with the initial population

$$X_0 = \{\text{EGG}, \text{SQUIRREL}, \text{SQUIRREL}, \text{TREE}\},$$

i.e., this time starting with two SQUIRRELS instead of only one. **Does this addition of another SQUIRREL affect the chances of a BUTTERFLY appearing, which you computed for task (c)? Give the chance and show how your reasoning.** (4pts)

Hint: Review exactly which behavior A uses.

No, the chances are not affected because A , as it is defined above, always chooses the *rule first*, which does not depend on the amount of particles available. Since we do not change the number of applicable rules, we also do not change the chance of a BUTTERFLY appearing.

6 Evolutionary Computing

15pts

If necessary, you can recollect the definition for evolutionary algorithms as used in the lecture on this page. (Hint: Try the tasks first before reading this lengthy definition again.)

Algorithm 2 (typical evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process.

- Let $\sigma_N^{survivors}, \sigma_N^{parents} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be (possibly randomized or non-deterministic) selection functions that return $N \in \mathbb{N}$ individuals, i.e., $|\sigma_N(X)| = N$ and $\sigma_N(X) \subseteq X$ for all X . They may possibly depend on $\mathcal{E}$ (most commonly ϕ). Let $\varrho_N^{mutants} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be a special selection function that, given a population X , returns a random subset $X' \subseteq X$ so that $|X'| = N$. Note that $\varrho_{|X|}(X) = X$ for all X .
- Let $mutate : \mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) single-individual mutation function. We write $mutate[_] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the per-individual application of $mutate$, i.e., $mutate[X] = \{mutate(x) : x \in X\}$.
- Let $recombine : \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) two-parent recombination function. We write $recombine[_] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the random-pairing application of $recombine$, i.e., $recombine[X] = \{recombine(x_1, x_2)\} \uplus recombine[X \setminus \{x_1, x_2\}]$ with $x_1, x_2 \sim X$ and $recombine[\{x\}] = \emptyset$ for all x as well as $recombine[\emptyset] = \emptyset$.
- Let $migrate : \emptyset \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) individual creation function. We write $migrate_N : \emptyset \rightarrow \wp^*(\mathcal{X})$ with $N \in \mathbb{N}$ for the iterated application of $migrate$, i.e., $migrate_N[] = \{migrate()\} \uplus migrate_{N-1}[]$ with $migrate_0[] = \emptyset$.

The process $\mathcal{E}$ continues via a typical evolutionary algorithm if E has the form

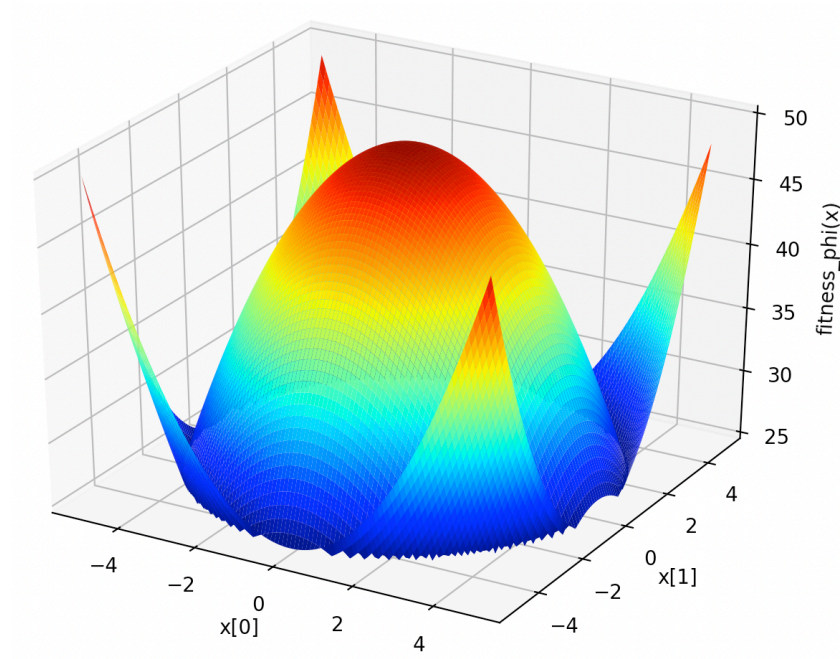
$$E(\langle X_u \rangle_{0 \leq u \leq t}, \phi) = X_{t+1} = \sigma_{|X_t|}^{survivors}(X_t \uplus mutate[\varrho_M^{mutants}(X_t)] \uplus recombine[\sigma_{2C}^{parents}(X_t)] \uplus migrate_H[])$$

where $M \in \mathbb{N}$ is the number of mutants produced per generation, $C \in \mathbb{N}$ is the number of children produced per generation, and $H \in \mathbb{N}$ is the number of migrants (sometimes called hypermutants) generated per generation. Especially M is often expressed as a rate for random choice within the population, i.e., $M = \lceil m \cdot |X_t| \rceil$ where $m \in \mathbb{R}$ is called mutation rate.

We now consider an evolutionary algorithm that searches through individuals within $\mathcal{X} = Q^2$ where $Q = [-5; +5] \subset \mathbb{R}$. It does so by minimizing a fitness function $\phi : Q^2 \rightarrow \mathbb{R}$, which is given via the following Python function:

```
def fitness_phi(x):
    return max(x[0]**2 + x[1]**2, 50 - x[0]**2 - x[1]**2)
```

The resulting solution landscape can be seen as shown below:



(a) This fitness function features multiple global optima. **Shortly describe a technique that should help us find multiple different global optima within one single run of the evolutionary algorithm.** (3pts)

We can measure the *diversity* single individuals contribute to the population and prefer individuals with more diversity during selection. This should help to spread out the population and possibly discover multiple optima.

(b) Consider the following two recombination functions given in Python. Note that the Python function `random.random()` returns a random floating point number n between 0.0 (inclusive) and 1.0 (exclusive), i.e., $n \sim [0.0; 1.0)$.

```
import random

def recombine_avg(parent_a, parent_b):
    child = [0.0, 0.0]
    child[0] = (parent_a[0] + parent_b[0]) / 2.0
    child[1] = (parent_a[1] + parent_b[1]) / 2.0
    return child

def recombine_xover(parent_a, parent_b):
    if random.random() < 0.5:
        return [parent_a[0], parent_b[1]]
    else:
        return [parent_b[0], parent_a[1]]
```

Which of the two recombination functions `recombine_avg` and `recombine_xover` is better suited for the optimization problem given by `fitness_phi`? Explain why. (3pts)

`recombine_avg` has a tendency generate new individuals near the center of the population. This makes it nearly useless during the start of evolution, as individuals will be spread out throughout the whole search space and the center region of the solution landscape has very bad fitness. Thus, `recombine_xover` is better suited.

(c) We define a *nice* mutation function to have the following qualities:

- It does not return solution candidates that lie out of the search space's boundaries.
- Its effects are random and not affected by fitness.
- Its effects work the same way in all the individual's dimensions.
- Its effects *completely* change the original individual *only very rarely at most*.
- There is *no point* within the search space that can *never* be reached via mutation, provided any arbitrary individual as a starting point and infinite time.

In a suitable formalism (Python, pseudo-code, mathematics, ...) give a *nice* mutation function for the above evolutionary algorithm. (5pts)

```
import random
import math

def mutate(parent):
    index = math.floor(random.random() * 2)
    parent[index] = parent[index] + random.random() - 0.5
    if parent[index] < -5:
        parent[index] = -5
    if parent[index] > 5:
        parent[index] = 5
    return parent
```

(d) Consider that we now construct an evolutionary algorithm with local search, i.e., we augment the evolutionary algorithm by using a greedy gradient-based search algorithm on each single individual before the fitness evaluation. **Given the fitness function `fitness_phi` (and thus the solution landscape above), what should we expect to happen when running this augmented evolutionary algorithm? As precisely as possible, explain why this setup is reasonable (or not) to optimize for the given fitness function `fitness_phi`?** (4pts)

Greedy gradient-based search on `fitness_phi` should immediately converge towards a global minimum. The setup thus yields results very quickly, but it does not use the evolutionary algorithm at all, which makes the overall setup unreasonable. We should instead use the greedy gradient-based search only.

7 Population-Based Optimization

5pts

We now consider a minimal case of a population-based optimization algorithm to help solve the previous solution landscape of Task 6. We still want to minimize the fitness function $\phi : Q^2 \rightarrow \mathbb{R}$ through individuals within $\mathcal{X} = Q^2$ where $Q = [-5; +5] \subset \mathbb{R}$, given as the slightly altered Python function:

```
def fitness_phi(x, y):  
    return max(x**2 + y**2, 50 - x**2 - y**2)
```

but this time we employ the following algorithm:

Algorithm 3 (particle pair algorithm (*PaPaiA*)). Let A, B be a pair of solution candidates defined by their position vectors $p_A, p_B \in Q^2$, each with velocities $v_A, v_B \in \mathbb{R}^2$ and memory of their best solution position $p_A^{\text{best}}(t), p_B^{\text{best}}(t)$ at time step t . We define $g^{\text{best}}(t) := \frac{1}{2}|p_A^{\text{best}}(t) + p_B^{\text{best}}(t)|$ as the average of the two best-solutions at each time step. Every step, the particles are updated via the functions

$$p_{i,d}(t+1) = p_{i,d}(t) + v_{i,d}(t+1)$$

and

$$v_{i,d}(t+1) = v_{i,d}(t) + \frac{1}{t}|g_d^{\text{best}}(t) - p_{i,d}^{\text{best}}(t)|$$

where $i \in \{A, B\}$ and $d \in \{1, 2\} \hat{=} \{\text{"X"}, \text{"Y"}\}$. Initially $p_i^{\text{best}} = p_i(0)$ and $v_i = (0, 0)$.

(a) **Briefly explain the functionality of this algorithm, as well as one advantage and one disadvantage when applied to `fitness_phi` as a target function.** (3pts)

The algorithm draws the particles together, depending on which of the two has the better position, i.e., the current g^{best} . This algorithm has the advantage of annealed attraction, meaning search will become more fine-granular as time goes on: As the velocities are additive, the trajectory course corrections will become smaller as time goes on. This can also turn out to be a disadvantage, as velocity updates at a later time will only have a very small effect and will not be able to adjust the particles direction appropriately.

(b) **Give two starting positions for $p_A(0), p_B(0)$ where the *PaPaiA* optimization will work well for `fitness_phi` as a target function. Briefly state why.** (2pts)

What would work well for `fitness_phi`, in particular, is having the particles start from opposite sides of the solution space (as they will move together), i.e., $[-5, -5]$ and $[5, 5]$. You could also let one of the particles start from the center and the other from one of the corners for a similar effect, i.e., $[-5, -5]$ and $[0, 0]$.